

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Answer

Servlet-based application architecture: A servlet web application mainly contains a client/browser, web server or servlet container such as Apache Tomcat, Java Servlet classes, and a database when persistent storage is required. The browser sends an HTTP request to the server. Tomcat receives the request, finds the required servlet, and calls the appropriate method such as doGet() or doPost(). The servlet processes the data and sends an HTTP response back to the browser.

Basic flow

Browser ® HTTP Request ® Tomcat/Servlet Container ® Servlet ® Processing/Database ® HTTP Response ® Browser

Servlet using GET and POST

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
        response.getWriter().println("GET received: " + name);
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");

        response.getWriter().println("Student Name: " + name);
        response.getWriter().println("Email: " + email);
    }
}
```

Servlet life cycle

- 1. init():** Called once when the servlet is created. It is used for initialization tasks.
- 2. service():** Called for each client request. It determines whether doGet(), doPost(), or another HTTP method should handle the request.
- 3. destroy():** Called once before the servlet is removed from service. It is used for cleanup tasks.

GET vs POST for sensitive data

POST is more appropriate for sensitive data such as passwords and personal registration information. GET places submitted values in the URL, where they may appear in browser history, bookmarks, logs, and other locations. POST sends the form data in the request body and is therefore preferable for form submission involving sensitive information. For real applications, HTTPS should also be used because POST by itself does not encrypt data.

Hello World!

My first Java web application is running on Tomcat.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Answer

Session management allows a web application to remember a user after login. HTTP is stateless, so each request is independent unless the application uses a session-tracking mechanism. HttpSession can store user information on the server, while a cookie can store a small identifier in the user's browser.

Login servlet using HttpSession

```
import java.io.IOException;
import      jakarta.servlet.ServletException;
import  jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import      jakarta.servlet.http.HttpServletRequest;
import      jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
```

```

        HttpServletResponse response)
        throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if (username.equals("admin") && password.equals("1234")) {

        HttpSession session = request.getSession();
        session.setAttribute("username", username);

        response.sendRedirect("HomeServlet");

    } else {
        response.getWriter().println("Invalid Login");
    }
}
}

```

Home servlet – checking the session

```

HttpSession session = request.getSession(false);

if (session != null &&
    session.getAttribute("username") != null)
{

    response.getWriter().println(
        "Welcome " + session.getAttribute("username"));

} else {
    response.sendRedirect("login.html");
}

```

Using a Cookie

```

Cookie cookie = new Cookie("username", "admin");
response.addCookie(cookie);

```

How session tracking works

1. The user logs in.
2. The server creates an HttpSession.
3. A session ID is associated with the user's browser, commonly through a cookie such as JSESSIONID.
4. The browser sends the session identifier with later requests.
5. The server uses the identifier to find the user's session data.
6. After logout, the session can be invalidated.

Login Page

Username:

Password:

Login

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer

JDBC (Java Database Connectivity) is an API used by Java applications to communicate with relational databases such as MySQL. In a web application, a servlet can receive product information from an HTML form, use JDBC to connect to MySQL, execute an INSERT statement, and later execute a SELECT statement to retrieve products.

JDBC architecture

Java Servlet/Application ® JDBC API ® MySQL JDBC Driver ® MySQL Database

The JDBC API provides classes and interfaces such as Connection, PreparedStatement, and ResultSet. The MySQL Connector/J driver translates JDBC calls into commands understood by MySQL.

Steps in JDBC database connectivity

- 1. Load/register the driver:** Make the MySQL Connector/J driver available to the application.
- 2. Establish connection:** Use `DriverManager.getConnection()` with the database URL, username, and password.
- 3. Create a statement:** Use `PreparedStatement` for parameterized SQL.
- 4. Execute SQL:** Use `executeUpdate()` for INSERT/UPDATE/DELETE and `executeQuery()` for SELECT.
- 5. Process the result:** Read rows from `ResultSet`.
- 6. Close resources:** Close `ResultSet`, `PreparedStatement`, and `Connection`, preferably using `try-with-resources`.

Database table

```
CREATE DATABASE shop;

USE shop;

CREATE TABLE product (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    price DOUBLE,
    quantity INT
);
```

Simple ProductServlet – Insert and Retrieve

```
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/ProductServlet")
public class ProductServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
        double price = Double.parseDouble(request.getParameter("price"));
        int quantity = Integer.parseInt(request.getParameter("quantity"));

        String url = "jdbc:mysql://localhost:3306/shop";
        String user = "root";
        String password = "root";

        String sql = "INSERT INTO product(name, price, quantity) " + "VALUES (?, ?, ?)";
```

```

        try            (Connection            con            =
                        DriverManager.getConnection
                        ( url, user, password);
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1,            name);
        ps.setDouble(2,            price);
        ps.setInt(3, quantity);
        ps.executeUpdate();

        response.getWriter().println(

                "Product added successfully");

        }        catch            (Exception            e)            {
                e.printStackTrace();
                response.getWriter().println(
                "Database Error: " + e.getMessage());
        }
}

protected void doGet(HttpServletRequest request,
                        HttpServletResponse            response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String url = "jdbc:mysql://localhost:3306/shop";
        String user = "root";
        String password = "root";

        String sql = "SELECT * FROM product";

        try            (Connection            con            =
                        DriverManager.getConnection
                        ( url, user, password);
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
        out.println("<h2>Products</h2>");

        while (rs.next()) {

                out.println(
                        rs.getInt("id") + " " +
                        rs.getString("name") + " " +
                        rs.getDouble("price") + " " +
                        rs.getInt("quantity") + "<br>");
        }

        }        catch            (Exception            e)            {
                e.printStackTrace();
                out.println("Database Error: " + e.getMessage());
        }
}
}

```

HTML form for ProductServlet

```
<form action="ProductServlet" method="post">
```

```
Product Name:
<input type="text" name="name"><br><br>

Price:
<input type="text" name="price"><br><br>

Quantity:
<input type="text" name="quantity"><br><br>

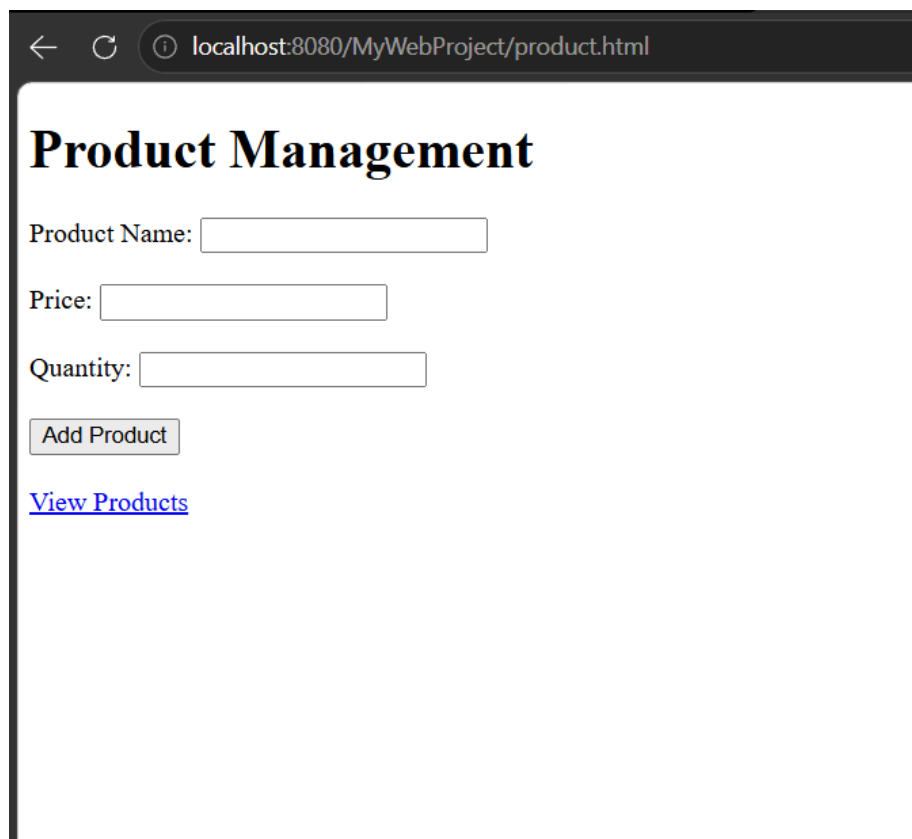
<input type="submit" value="Add Product">

</form>

<a href="ProductServlet">View Products</a>
```

Exception handling

JDBC operations can fail because of an incorrect database URL, wrong username/password, missing JDBC driver, invalid SQL, or database connection problems. The try-catch block handles these exceptions and prevents the application from terminating unexpectedly. Try-with-resources automatically closes JDBC resources after use.



← ↻ ⓘ localhost:8080/MyWebProject/product.html

Product Management

Product Name:

Price:

Quantity:

[View Products](#)

Code of Conduct:

I Ch. Bhargav Ram (192425334) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand